

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

Projeto de Arquitetura

O que é Arquitetura?

O que é Arquitetura?

Quando consideramos um edifício, a arquitetura é:

- Como os vários componentes do edifício são integrados para formar um todo coeso.
- O modo como o edifício se ajusta ao seu ambiente e se integra a outros edifícios próximos.
- O grau com que o edifício atende a seu propósito declarado e satisfaz às necessidades de seu proprietário.
- A maneira como texturas, cores e materiais são combinados para criar a fachada e o impacto visual do edifício.
- A lista é interminável.

O que é Arquitetura?

A arquitetura não é o software operacional, é uma representação que nos permite:

1. analisar a efetividade do projeto no atendimento dos requisitos declarados;
2. considerar alternativas de arquitetura em um estágio em que fazer mudanças de projeto ainda é relativamente fácil; e
3. reduzir os riscos associados à construção do software.

O que é Arquitetura?

A IEEE 1471 define arquitetura como:

“the fundamental organization of a system embodied in its components, their relationships to each other and to the environment, and the principles guiding its design and evolution.”

O que é Arquitetura?

De acordo com Martin Fowler (Who Needs an Architect?), Ralph Johnson, um dos revisores da definição, escreveu em uma lista de e-mails:

“[...] esta claramente é uma definição completamente falsa.”

O que é Arquitetura?

Segundo Ralph Johnson:

- Os clientes têm um conceito diferente dos desenvolvedores
- Os clientes não se importam com a estrutura dos componentes significativos
- O que torna um componente significativo?
- Ele é significativo porque os desenvolvedores especialistas assim o afirmam.
- Um dos fatores é a dificuldade do componente ser alterado.
- Por isso, uma arquitetura é o conceito mais elevado que os desenvolvedores têm de um sistema em seu ambiente.

O que é Arquitetura?

In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This shared understanding is called architecture.

Martin Fowler (2003)

O que é Arquitetura?

Esse entendimento inclui:

- Como o sistema é dividido em componentes.
- Como esses componentes interagem através de interfaces.

O que é Arquitetura?

Por exemplo, em aplicações empresariais, o que os desenvolvedores especialistas tendem a considerar que é crucial?

O que é Arquitetura?

Por exemplo, em aplicações empresariais, o que os desenvolvedores especialistas tendem a considerar que é crucial?

Persistência!

"Nós usamos Oracle e temos nossa própria camada de persistência para mapear os objetos para o BD".

O que é Arquitetura?

Essa é a ÚLTIMA, prometo!

O que é Arquitetura?

Architecture is about the important stuff. Whatever that is.

Ralph Johnson

Por que a arquitetura é importante?

- Fornece uma representação que facilita a comunicação entre todos os envolvidos.
 - Constitui um modelo relativamente pequeno e compreensível de como o sistema é estruturado e como seus componentes interagem.
- Destaca desde o início as decisões de projeto que terão um profundo impacto no trabalho de engenharia de software que se segue.

Por que a arquitetura é importante?

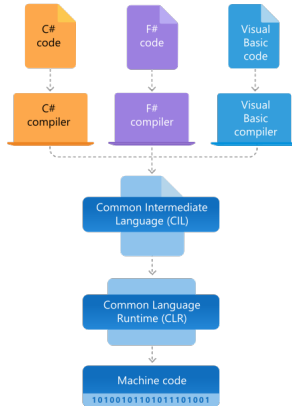


Figura 1: Arquitetura do .NET framework.

- As decisões de arquitetura (entre uma variedade de possibilidades) podem ser consideradas uma visão de arquitetura.
- **Para desenvolvedores ágeis:**
 - Um *Architectural Decision Record* (ADR) poderia conter apenas um título, um contexto, a decisão, o status e as consequências.
- Projeto dominante: arquitetura ou processo de software inovador que se torna **padrão** após período de adaptação e uso bem-sucedido no mercado.

- O arquiteto de software junto com o *product owner*:
 - Confronta histórias de usuários arquiteturais vs histórias de usuários de negócio.
 - As equipes têm liberdade para fazer mudanças à medida que surgem novos requisitos.
 - O produto em evolução deve ser aprovado conforme cada 'protótipo' é concluído.

Por que a arquitetura é importante?

Martin Fowler:

<https://youtu.be/DngAZyWMGR0?si=4bqdDdSo1ldBsnGC>

Estilos de Arquitetura

Procure no Google por imagens com as seguintes expressões:

- *American colonial style with central hall.*
- *Prefabricated house on an A-shaped wooden frame.*
- **O software também apresenta um estilo de arquitetura.**

Cada estilo descreve uma categoria de sistema:

- um conjunto de componentes (por exemplo, um banco de dados, módulos computacionais) que realiza uma função exigida por um sistema;
- um conjunto de interfaces que habilitam a “comunicação, coordenação e cooperação” entre os componentes;
- restrições que definem como os componentes podem ser integrados para formar o sistema;

Diferença entre estilo arquitetural e padrão de arquitetura (Pressman):

- Um estilo arquitetural é estabelecer uma estrutura para todos os componentes do sistema.
- O escopo de um padrão é menos abrangente, concentrando-se em um aspecto da arquitetura e não na arquitetura em sua totalidade.

Embora milhões de sistemas computacionais tenham sido criados nos últimos 60 anos, a vasta maioria pode ser classificada em um número relativamente pequeno de estilos de arquitetura.

Assim que a **engenharia de requisitos** revelar as **características e restrições do sistema** a ser construído, o **estilo e/ou combinação de padrões de arquitetura** que melhor se adequar pode ser escolhido.

Em muitos casos, mais de um padrão poderia ser apropriado, e estilos de arquitetura alternativos podem ser projetados e avaliados.

Por exemplo, um estilo em camadas (apropriado para a maioria dos sistemas) pode ser combinado com uma arquitetura centralizada em dados em diversas aplicações de bancos de dados.

Centralizada em Dados

- Um **repositório de dados** reside no centro dessa arquitetura
- Em geral, os componentes acessam esses dados para:
 - editar
 - incluir
 - eliminar
 - modificar

Centralizadas em Dados

A Figura 2 ilustra um estilo centralizado em dados típico.

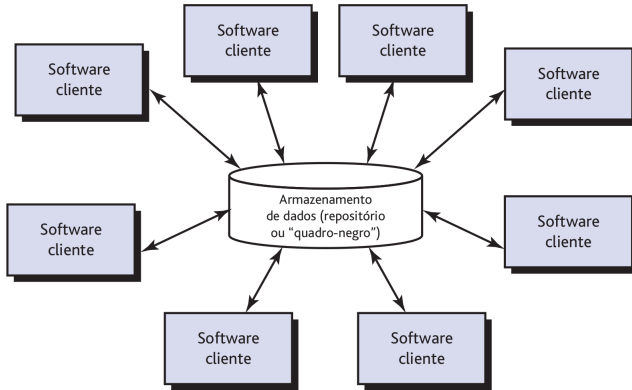


Figura 2: Arquitetura centralizada em dados.

Centralizadas em Dados

- Sub-sistemas necessitam trocar dados entre si. Essa troca pode acontecer de duas maneiras:
 - Compartilhamento através de um banco de dados ou repositório comum a todos os componentes.
 - Cada componente mantém o seu próprio banco de dados e transfere os dados explicitamente uns aos outros.
- Quando é necessário compartilhar um grande volume de dados, o modelo de repositório centralizado é mais utilizado.
- As arquiteturas centralizadas em dados promovem a integrabilidade:
 - Componentes existentes podem ser alterados e novos componentes clientes acrescentados à arquitetura sem se preocupar com outros clientes.

Vantagens:

- Proporciona um modo eficiente de compartilhamento de um grande volume de dados.
- Os componentes não necessitam conhecer a maneira como os dados são produzidos ou gerenciados (backup, segurança, etc. são transparentes).
- O modelo de compartilhamento pode ser definido unicamente pelo repositório.

Cliente-Servidor

- Componentes: são agrupados de acordo com a sua localização física: cliente ou servidor.
- Conectores: pontes de comunicação entre os componentes clientes e os componentes servidores (normalmente comunicação remota).
- Restrições: os clientes não são unidades autônomas, dependendo explicitamente dos servidores.

É um estilo voltado para sistemas distribuídos, que mostra como os dados e o processamento é dividido entre um conjunto de componentes.

Um conjunto de servidores oferecem serviços específicos, tais como impressão, gerenciamento de dados, etc.

Um conjunto de clientes podem executar esses serviços.

A comunicação entre eles se dá normalmente através de uma rede remota.

Vantagens:

- A distribuição dos dados é feita de uma forma direta e natural.
- Torna o uso de redes de computadores uma atividade efetiva.
- É escalável, isto é, fácil de adicionar novos servidores ou atualizar os já existentes.

Cliente-Servidor

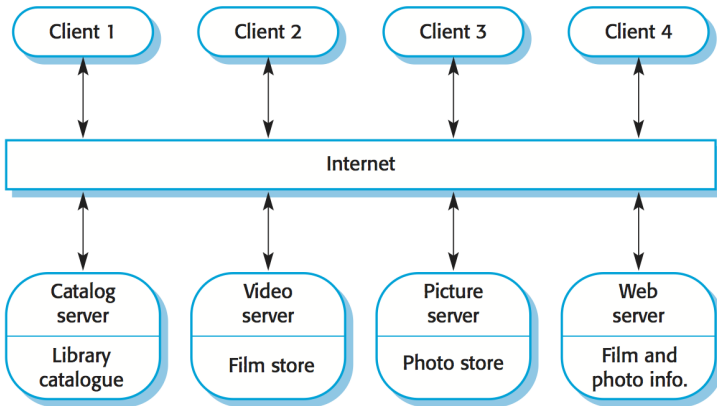


Figura 3: Arquitetura cliente-servidor de uma biblioteca de filmes.

Fluxo de Dados

- Essa arquitetura se aplica quando **dados de entrada** devem ser **transformados** por meio de uma série de componentes computacionais ou de manipulação em **dados de saída**.
- Um padrão **tubos-e-filtro** tem um conjunto de componentes, denominado filtros, conectados por tubos que transmitem dados de um componente para o seguinte.

- Cada filtro trabalha de modo independente.
- É projetado para esperar a entrada de dados de determinada forma e produz saída de dados da forma especificada.
- Um filtro não precisa conhecer o funcionamento interno de seus filtros vizinhos.
- Se o fluxo de dados ocorre em uma única linha de transformações, ele é denominado sequencial por lotes.
- Essa estrutura aceita um lote de dados e aplica uma série de componentes sequenciais (filtros) para transformá-lo.

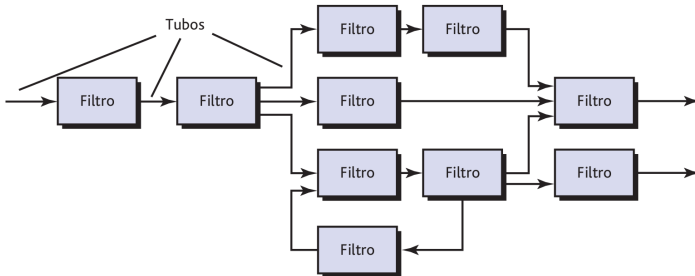


Figura 4: Arquitetura de fluxo de dados.

Fluxo de Dados

Na Figura 5 é apresentado um exemplo da arquitetura fluxo de dados (duto e filtro).

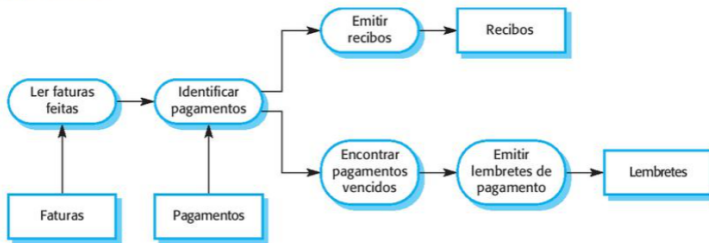


Figura 5: Arquitetura de fluxo de dados.

Camadas

Na arquitetura em **camadas**, são definidas várias camadas diferentes:

- Na camada mais externa, os componentes atendem às operações da interface do usuário.
- Na camada mais interna, fazem a interface com o sistema operacional.
- As camadas intermediárias fornecem serviços utilitários e funções de software de aplicação.

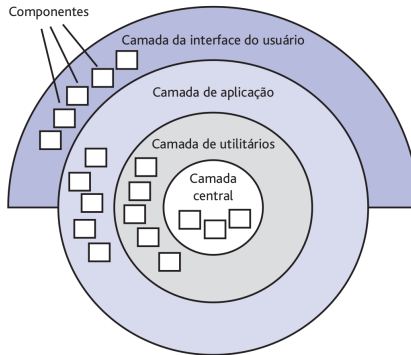


Figura 6: Arquitetura em camadas.

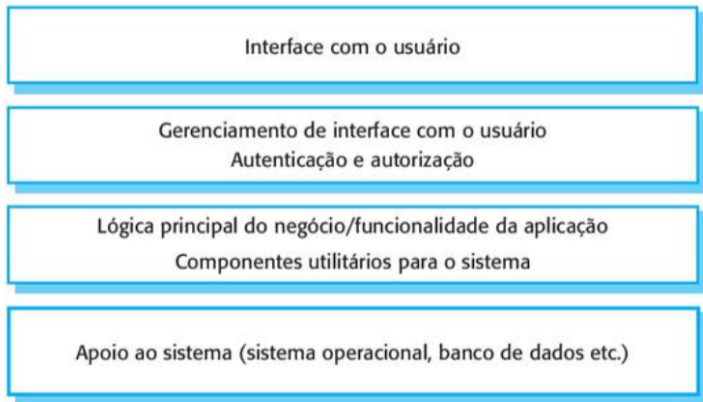


Figura 7: Uma arquitetura genérica em camadas.

Microserviços

Suponha que um sistema tenha sido particionado em módulos $(M_1, M_2, \dots, M_n)$, como mostrado na figura a seguir:

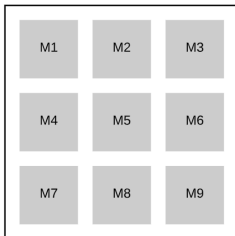


Figura 8: Um sistema particionado em 9 módulos.

- Em tempo de execução esses módulos serão executados pelo sistema operacional como um processo único.
- Portanto, compartilham o mesmo espaço de endereçamento.
- Isto é, o sistema é um grande monolito em tempo de execução.
- A chance de que mudanças em um módulo comprometa o funcionamento de outro módulo é grande.
- Apesar do desenvolvimento ser ágil, o processo de testes ainda é lento e burocrático.
- Para resolver esse gargalo, empresas passaram a adotar uma arquitetura baseada em microserviços

Microserviços são uma abordagem de arquitetura para a criação de aplicações, onde **cada pedaço dessa aplicação** é desenvolvido e disponibilizado de forma **independente**.

- Cada **processo** da aplicação é executado como um **serviço**.
- Quando falamos em microserviços, estamos nos referindo a uma funcionalidade que pode ser dividida em partes menores.
- Desse modo, essas pequenas partes se comunicam por meio de uma interface bem definida, por exemplo APIs.

- Como são executados de forma independente, cada serviço pode ser atualizado e implantado para atender às demandas de uma aplicação.
- Se forem construídos corretamente, os serviços independentes não afetarão uns aos outros: se um deles falhar, o restante da aplicação permanecerá em funcionamento.

Microserviços

A figura a seguir mostra os nove módulos do exemplo anterior sendo executados em 6 processos ou como 6 **microserviços** diferentes:

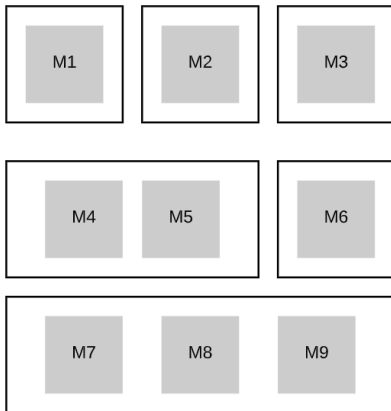


Figura 9: Nove módulos de um sistema sendo executados como 6 microserviços.

Microserviços

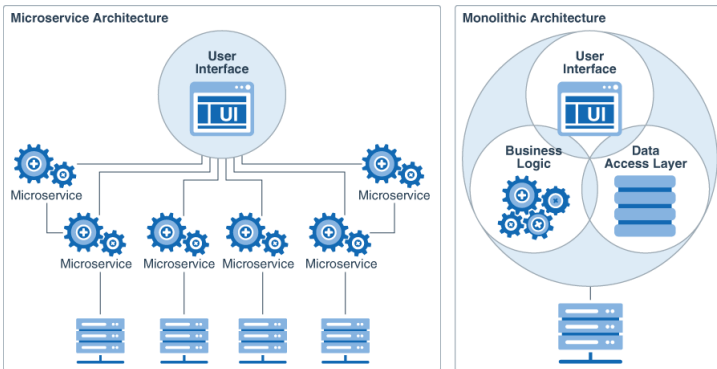


Figura 10: Arquitetura de microserviços e arquitetura monolítica.

Microserviços

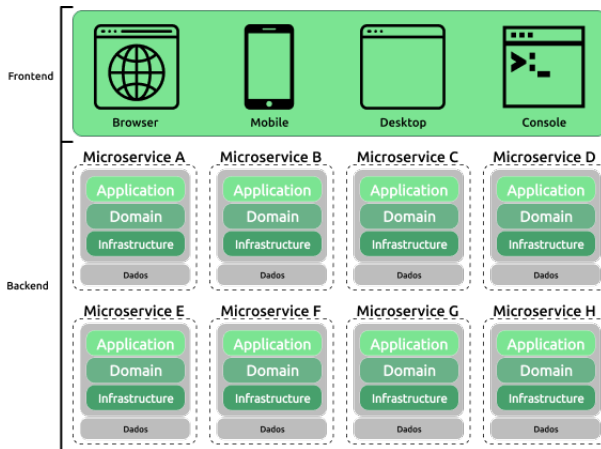


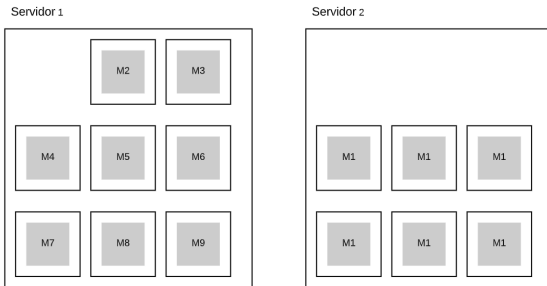
Figura 11: Arquitetura de microserviços.

Vantagens:

- **Agilidade:** com os deploys e testes independentes, temos uma maior agilidade no desenvolvimento e na implantação;
- **Baixo acoplamento:** com as aplicações são independentes, elas não possuem um acoplamento forte entre si. Isso facilita processos de manutenção, implantação e monitoramento;
- **Flexibilidade para implantação de tecnologias heterogêneas:** como não existe acoplamento explícito e cada serviço é independente, podemos ter diferentes serviços escritos em diferentes tecnologias comunicando-se entre si.

Vantagens:

- **Escalabilidade flexível:** cada serviço e funcionalidade pode ser escalado de maneira mais adequada e granular.
 - Com isso, é possível obter até mesmo economia na manutenção da infraestrutura;
- Suponha que o microserviço que contém M_1 seja responsável pela autenticação de usuários. Esse microserviço pode ser escalado horizontalmente:



Model-View-Controller

A arquitetura MVC define que uma aplicação pode ser organizada em três camadas (grupos):

- O **modelo** armazena todos os dados manipulados pela aplicação e a lógica de processamento específica.
- A **visão** contém todas as funções específicas à interface e possibilita a apresentação do conteúdo e lógica de processamento exigido pelo usuário;
- O **controlador** interpreta os eventos gerados e gerencia o acesso ao modelo e à visão e coordena o fluxo de dados entre eles.

Model-View-Controller

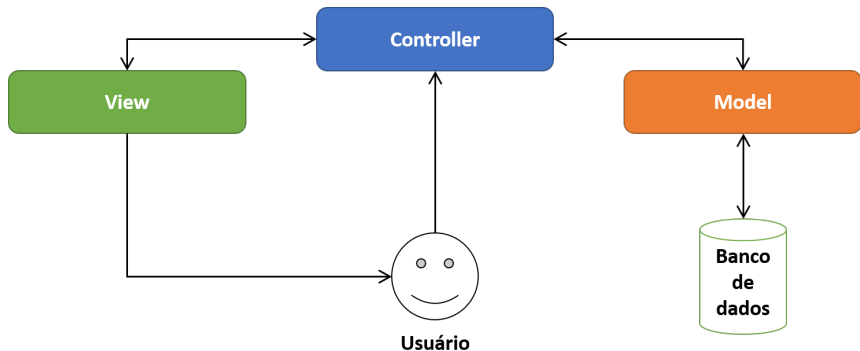


Figura 12: Diagrama do padrão MVC.

Model-View-Controller

- Porém, em muitos sistemas não existe uma distinção clara entre Visão e Controladores:

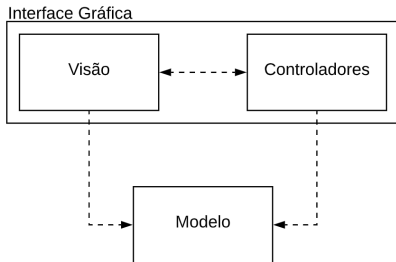


Figura 13: Arquitetura MVC sem a distinção clara entre Visão e Controladores.

Vamos utilizar o exemplo de uma página web, onde o usuário pode realizar o cadastro de clientes.

- Com a popularização da Web, apareceram frameworks para implementação de sistemas Web que se denominaram frameworks MVC.
- Como exemplo, podemos citar Spring (para Java), Ruby on Rails, Django (para Python) e CakePHP.
- Esses frameworks expandiram e adaptaram o conceito de MVC para Web.

Como mencionamos antes, esses frameworks forçam a organização de um sistema Web em três partes:

- Visão, composta por páginas HTML;
- Controladores, que processam uma solicitação e geram uma nova visão como resposta;
- Modelo, que é a camada que persiste os dados em um banco de dados.

Model-View-Controller

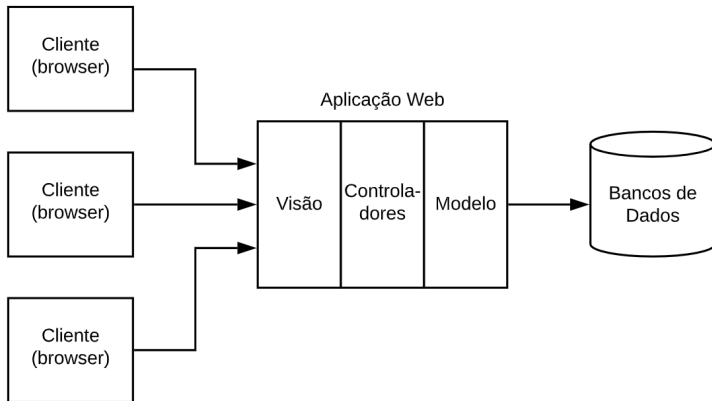


Figura 14: Arquitetura MVC Web.

Model-View-Controller

Vantagens do MVC:

- Favorece a especialização do trabalho de desenvolvimento: front-end, back-end, DBA.
- Favorece testabilidade: é mais fácil testar objetos não-visuais (visão).
- Permite que classes de Modelo sejam usadas por diferentes Visões:

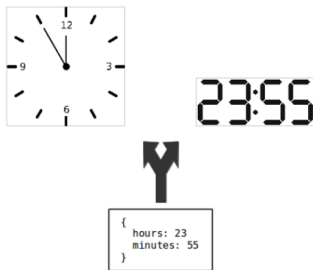


Figura 15: Sistema MVC com mais de uma visão.

Arquiteturas Orientadas a Mensagens

- A comunicação entre clientes e servidores é mediada por um serviço.
- Esse serviço provê uma **fila de mensagens**.
- A comunicação pelo lado do cliente é **assíncrona**:
 - Assim que a mensagem é inserida, cliente está liberado para continuar processamento.

Orientadas a Mensagens

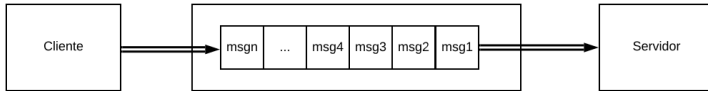


Figura 16: Arquitetura Orientada a Mensagens.

Orientadas a Mensagens

- Clientes inserem mensagens na fila (produtores).
- Servidores retiram mensagens da fila (consumidores).
- Nessa aplicação distribuída, clientes e servidores:
 - Não precisam se conhecer (desacoplamento no espaço).
 - Não precisam estar simultaneamente disponíveis para se comunicarem (desacoplamento no tempo).
- Isso permite escalar mais facilmente um sistema distribuído.

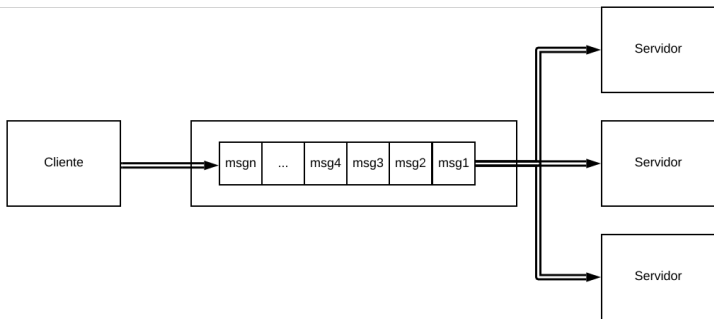


Figura 17: Arquitetura Orientada a Mensagens.

Publish/Subscribe

Na arquitetura **Publish/Subscribe**, há dois componentes (papéis) principais:

- Publicadores
- Assinantes

Os publicadores e assinantes se comunicam por meio de mensagens denominadas de **eventos**.

- Publicadores: produzem eventos e os publicam no serviço de publish/subscribe
- Assinantes: Os assinantes devem assinar os eventos de seu interesse

Quando um evento é publicado, os seus assinantes são notificados.

Em publish/subscribe, os assinantes são notificados assincronamente.

Publish/Subscribe

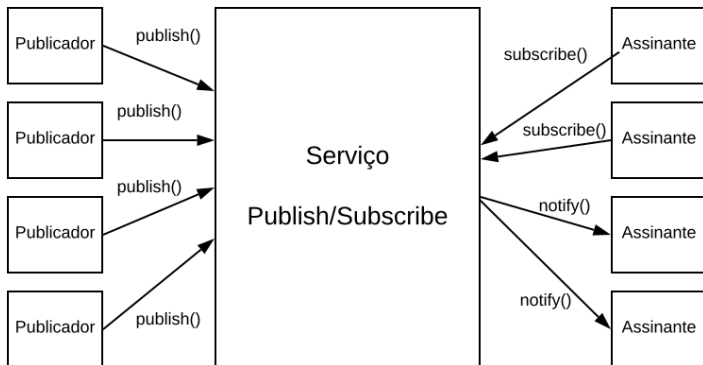


Figura 18: Arquitetura Publish/Subscribe.

Exemplo de uma companhia aérea:

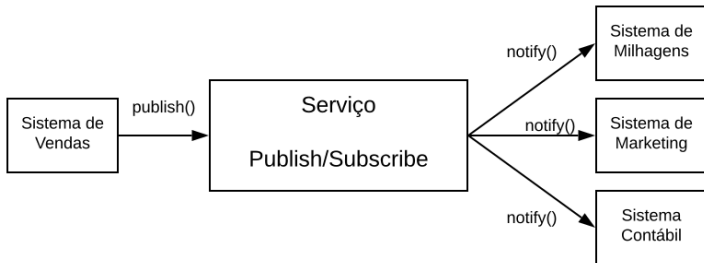


Figura 19: Arquitetura Publish/Subscribe.

Essa arquitetura tem as seguintes características:

1. comunicação em grupo, pois o mesmo evento é assinado por três sistemas;
2. desacoplamento no espaço, pois o sistema de vendas não tem conhecimento dos sistemas interessados nos eventos que ele publica;
3. desacoplamento no tempo, pois o sistema de publish/subscribe reenvia os eventos caso os sistemas assinantes estejam fora do ar;
4. notificação assíncrona, pois os assinantes são notificados assim que um evento ocorre; isto é, eles não precisam consultar periodicamente o sistema publish/subscribe sobre a ocorrência dos eventos de interesse.